

퀄리저널 키워드 뉴스 시스템 문제해결보고서 (v1.0)

작성일: 2025 - 10 - 05 (KST)

목표: 인수인계보고서 및 관련 계획/고도화 문서를 종합 분석하여, 현 시스템의 미흡/리스크를 도출하고 즉시 실행 가능한 개선 방안과 단계별 실행 가이드를 제시한다.

1. 요약 (Executive Summary)

- 핵심 병목: JSON 수동 편집 · 승인(협업 취약), 정적 필터(커뮤니티 0건/과도한 노이즈), RSS 피드 변경 대응 부재, Windows PowerShell 의존, 검색/탐색 취약.
- 품질 유지: 발행 품질 게이트(승인 ≥ 15)를 유지하되, 수집 · 평가(QG/FC)와 스코어링 자동화를 복원 · 강화하여 ‘사람은 최종 승인’ 구조로 부담을 경감.
- 운영 UI: FastAPI + React/Vue 기반 관리자 UI를 신설해 ‘키워드 입력→수집→후보검토→승인→발행’ 전 과정을 한 화면에서 작업.
- 데이터 기반: SQLite→PostgreSQL(선택)로 이관 후 Elasticsearch 연동을 검토하여 검색 · 타임라인 · 연관탐색을 강화.
- 신뢰도/안정성: 피드 모니터링 스크립트, 동적 임계값(커뮤니티), 에러/품질 로그, 아카이브 네이밍 규칙으로 운영 리스크를 선제 관리.
- 5일 플랜: D1 스키마/백업 정리, D2 수집기/동의어/커뮤니티 필터 최적화, D3 QG/FC · 스코어링 복원, D4 요약/번역 · 발행 템플릿 정비, D5 관리자 UI 베타 오픈.

2. 현재 파이프라인과 운영 정책 (Baseline)

퀄리저널은 ‘하루 한 키워드’를 중심으로 수집→품질평가(QG/FC)→편집자 승인→발행의 흐름을 갖는다. 작업본 `selected_keyword_articles.json`과 발행본 `selected_articles.json`을 구분 관리하며, 발행 시 아카이브(YYYY-MM-DD_KEYWORD.{json,md,html})로 저장한다. 품질 게이트로 승인 기사 ≥ 15 를 요구한다. 자동화는 PowerShell 스크립트(`run_quali_today.ps1`)가 orchestrator를 호출해 수집 · 재구성 · 보충 · 동기화 · 발행을 순차 실행한다.

3. 미흡 · 부족 · 리스크 진단

- 편집/협업: JSON 수동 편집과 승인(`approved/editor_note`) 중심이라 실수 · 충돌 위험이 높고 협업 추적이 어렵다. 관리자 UI가 제한적이거나 부재.
- 품질/선별: QG/FC/스코어링이 비활성/단순화된 경향. 승인 ≥ 15 충족을 위해 수작업이 과다해지면 긴급모드(승인 무시) 유혹이 커짐.
- 커뮤니티 필터: `score_threshold` · 최소 댓글/추천수 등 임계값이 정적으로 고정되어 ‘0건’ / ‘잡음 과다’ 상황이 빈발.
- 소스 안정성: RSS/뉴스 피드 URL 변경이나 404 발생 시 자동 검증/알림 부재로 수집 실패 위험.
- 운영환경: Windows PowerShell 중심 자동화는 f-string · 경로/인코딩 이슈, 플랫폼 종속성 문제를 야기.
- 검색/탐색: JSON 파일 중심 저장은 빠른 검색 · 연관 탐색 · 타임라인 제공에 제약.
- 이력 · 통계 · 지표 부족.
- 아카이브: 동일 키워드 재발행 시 파일 덮어쓰기/중복 리스크. 백업 · 용량 관리 정책 미흡.

4. 개선 방안 (Solution Design)

4.1 관리자 UI/협업

- FastAPI 백엔드 + React/Vue 프론트로 ‘ 키워드 뉴스 ’ 전용 탭 신설: 후보 리스트, 점수/배지(QG/FC), 승인 토글, 코멘트 입력, 미리보기, 발행 버튼.
- 키보드 단축키(↑↓, Enter 승인), 상태/점수 필터, 검색창, 변경이력(감사 로그) 및 동시 편집 경고.
- API 예시: GET /api/keyword/{kw}, POST /api/keyword/{kw}/collect, POST /api/keyword/{kw}/approve, GET /api/preview/{kw}, POST /api/publish/{kw}.

4.2 수집기/필터 고도화

- 키워드 동의어(keyword_synonyms.json) 적용 및 제목 · 요약 · 본문 빈도 가중치.
- 커뮤니티 동적 임계값: 최근 7일 점수 분포의 $P50 \pm \sigma$ 기반으로 score_threshold 자동 추천(운영자가 UI에서 최종 확정).
- 피드 센티넬 스크립트: official_sources.json을 일일 점검(HTTP status/구조 변화) · 알림.

4.3 QG/FC/스코어러 복원

- QG: 본문 글자수/문장수/링크수 + 금지요소(로그인/구독유도/오류페이지) 필터. 섹션별 가변 임계값.
- FC: 숫자 · 연도 · 표준코드 · 공식 문서 링크 탐지로 신뢰도 가중.
- 도메인/키워드/중복이슈 스코어 합산(total_score) 규칙을 editor_rules.json에 선언적으로 관리.

4.4 저장소/검색

- 초기 SQLite → 필요시 PostgreSQL로 이관. 테이블: keywords, documents, scores, approvals.
- ElasticSearch 연동(선택): 타임라인/유사검색/벡터검색으로 ‘ 역사 큐레이션 ’ 탐색성을 강화.

4.5 운영/품질/안정성

- 발행 품질 게이트(승인 ≥ 15) 상시 유지. ‘ force ’ 는 오직 운영자 권한 + 사후 자동 복구.
- 아카이브 네이밍: YYYY-MM-DD_HHMM_KEYWORD.{md,html,json} 규칙으로 중복 방지.
- 로깅/지표: 수집 성공률, 후보/승인 건수, QG/FC 탈락률, 발행 소요시간 · 오류율 대시보드화.

4.6 성능/배포

- 크롤링 aiohttp/asyncio 병렬화, 리트라이/백오프.
- Docker 이미지 · CI/CD, 크로스플랫폼 스케줄러(cron/GitHub Actions)로 PowerShell 의존 축소.

5. 적용 예시 (구성/코드 스니펫)

커뮤니티 동적 임계값 추천 (파이썬 예시)

```
import numpy as np
```

```
def recommend_threshold(scores, base=3.8, k=0.5):
```

```
    scores = np.array(scores)
```

```
    if len(scores) == 0:
```

```
        return base
```

```
    p50 = np.percentile(scores, 50)
```

```
    sigma = scores.std()
```

```
    return round(max(0.0, p50 - k*sigma), 2)
```

사용: 최근 7일 커뮤니티 점수 목록으로 추천값 계산 → UI에서 제안

RSS 피드 센티넬 (개념 예시)

```
import requests, json, time
```

```
def probe_feed(url, timeout=10):
```

```
    r = requests.get(url, timeout=timeout)
```

```
    return {"url": url, "status": r.status_code, "ok": r.ok}
```

```
# 매일 official_sources.json 순회 → 200 외/패턴변화 감지시 알림
```

```
# 스코어 합산 규칙 (editor_rules.json 일부 예시)
```

```
{
    "domain_weights": {"ipc.org": 8, "iconnect007.com": 6, "reddit.com": 1},
    "keyword_weights": {"J - STD - 001": 3, "IPC - A - 610": 3, "reflow": 2},
    "score_formula": "total = kw + domain + fc + bonus"
}
```

6. 실행 가이드 — 5일 집중 플랜

D1 — 스키마 · 백업 · 규칙 정리

- 아카이브/선택 JSON 일괄 점검: repair_selection_files.py 실행, 구조 통일.
- 아카이브 네이밍 규칙 적용(YYYY-MM-DD_HHMM_KEYWORD.*); 전체 백업 스냅샷.
- config.json 정돈: features.require_editor_approval=True 재확인.

D2 — 수집기 · 동의어 · 커뮤니티 필터

- collect_news/papers/standards/community 점검; keyword_synonyms.json 반영.
- 커뮤니티 동적 임계값 추천 함수 연결 → UI에서 한 번에 적용 버튼.
- 피드 센티넬 스크립트 배치(일 1회).

D3 — QG/FC/스코어러 복원

- evaluate.py에서 QG(길이/문장/링크/금지요소)와 FC(근거요소 탐지) 재가동.
- editor_rules.json에 도메인 · 키워드 · 중복이슈 가중치 테이블 정리.
- 상위 20 자동 승인(force_approve_top20.py)은 운영자 전용으로 제한.

D4 — 요약/번역 · 발행

- translate_summarize.py 한국어 요약(2~3문장) + Glossary 적용.
- publish_engine 템플릿 정리(섹션/타임라인/편집자 한마디).
- orchestrator - - publish - keyword 연동 검증.

D5 — 관리자 UI 베타

- 키워드 뉴스 탭: 목록/검색/필터/승인/코멘트/미리보기/발행.
- 변경이력 · 권한 · 단축키 적용. 베타 운영 후 피드백 반영.

7. KPI 및 리스크 관리

- KPI-1: 후보 대비 승인 비율($\geq 50\%$) · QG/FC 통과률($\geq 70\%$).
- KPI-2: 편집 처리 시간(목표: 30분↓) · 발행 소요(목표: 10분↓).
- KPI-3: 커뮤니티 ‘0건’ 발생률(목표: 0%) · 수집 실패 피드 비율(목표: $< 2\%$).
- KPI-4: 중복 발행/덧어쓰기 0건.

주요 리스크 및 대응:

- 외부 소스 변경/차단 → 센티넬/백오프/대체 소스 운영.
- API 키/요금 이슈 → 발행 시점 요약만 호출, 캐싱, 비용 모니터링.

- 긴급 발행 남용 → 운영자 권한/감사 로그/자동 복구 정책.

부록 A. 초딩 버전 7단계 체크리스트

- 1) 키워드 정하기 → 기록하기.
- 2) 수집 버튼 누르기(키워드 뉴스 모드).
- 3) 후보 기사에서 좋은 것 15개 체크하기.
- 4) 한 줄 코멘트 쓰기.
- 5) 미리보기로 모양 확인하기.
- 6) 발행 버튼 누르기.
- 7) 결과 링크 저장하기(아카이브 확인).

부록 B. 참고 문서(일부)

- 1005_1 웰리저널 프로젝트 인수인계 보고서.pdf
- 1004_1B 웰리저널 개발 계획서 (실전 중심 가이드).pdf
- 1004_2A 고도화 report.pdf
- 1004_3A 작업계획 report.pdf
- 1004_4 QualiNews_tech_assessment_5day_plan_2025 - 10 - 04.pdf
- 1005_1A 개선방안 report.pdf